

Report conversation

Follow these prompts to complete the task of writing the javascript function "math.factorial" using recursion.

INITIAL_PROMPT=You are an agent that writes JavaScript functions.\n+
Before writing any code think step by step about what you want to implement.\n+
Call the writeFunction function to submit the code of your JavaScript function.\n+
If the first try doesn't succeed, try again. Do not create mock functionality.\n;

```
export const GOAL_PROMPT = (namespace: string,  
description: string, args: string) =>
```

```
  Your task is to write the body of an async  
  JavaScript function.\nFunction namespace:
```

```
"${namespace}"\nArguments:
```

```
${args}.\nDescription: "${description}"\n+
```

```
  You must refer to function arguments as if  
  they were locally defined variables,  
  remember you're writing just the body of  
  the function.\n+
```

```
  Use only the function arguments above, do  
  not add new ones.\n+
```

```
  Since you are writing the body of the  
  function, remember to use the return  
  keyword if needed.\n+
```

```
  When using libraries, use the require  
  function to import them.\n+
```

```
  Do not require libraries aside from 'fs'  
  and 'axios'\n+
```

```
  Do not use external APIs that require  
  authentication or an API key.\n+
```

```
  Do not recursively call the "${namespace}"  
  function.\n+
```

```
  Example function body:\n+
```

```
const fs = require('fs');\n+\nreturn fs.readFileSync(path, encoding);\n;
```

Schema:

json

```
{\n  "name": "math.factorial",\n  "description": "Calculate the factorial of",\n  "arguments": "{ number: number }",\n  "code": "./math.factorial.js"\n}
```

You are an agent that writes JavaScript functions.

Before writing any code think step by step about what you want to implement.

Call the writeFunction function to submit the code of your JavaScript function.

If the first try doesn't succeed, try again. Do not create mock functionality.

Your task is to write the body of an async JavaScript function.

Function namespace: "math.factorial"

Arguments: { number: number }.

Description: "Calculate the factorial of a number"

You must refer to function arguments as if they were locally defined variables, remember you're writing just the body of the function.

Use only the function arguments above, do not add new ones.

Since you are writing the body of the function, remember to use the return keyword if needed.

When using libraries, use the require function to import them.

Do not require libraries aside from 'fs' and 'axios'

Do not use external APIs that require authentication or an API key.

Do not recursively call the "math.factorial" function.

Example function body:

```
const fs = require('fs');\nreturn fs.readFileSync(path, encoding);
```

Given the constraints, we need to write a function that calculates the factorial of a number using recursion. However, we are explicitly told not to recursively call the "math.factorial" function. This means we should create an inner helper function for the recursion.

Let's write the function:

javascript

 Copy code

```
// Check if the number is valid and non-negative
if (number < 0) {
    throw new Error("Number must be non-negative");
}

// Define the recursive helper function
function factorialHelper(n) {
    if (n === 0) {
        return 1;
    }
    return n * factorialHelper(n - 1);
}

// Call the helper function with the provided number
return factorialHelper(number);
```

This function first checks if the provided number is non-negative. If it is negative, it throws an error. Then, it defines a helper function `factorialHelper` that calculates the factorial using recursion. Finally, it calls this helper function with the provided number and returns the result.